

Group 10 What To Eat!!

Sylvey Lin, Ming-Lun Chen, Yi-Han Huang, Zih-Yi Cao

Problem Definition

Deciding what to eat has become a surprisingly time-consuming task for busy people. A quick check of the fridge often reveals forgotten ingredients, yet many still end up buying duplicates and tossing spoiled food later. When cooking feels uninspiring, they turn to restaurant apps only to scroll through countless listings without making a choice. The result is wasted money, unnecessary food waste, and daily frustration.

Main pain points

1. Forgotten fridge contents lead to repeated purchases and avoidable waste.
2. Lack of recipe ideas makes it hard to turn existing ingredients into meals.
3. An overload of restaurant options creates decision fatigue and steals time.

What to Eat offers a single solution to these problems. The application lets users log fridge items with minimal effort, tracks expiration dates, and highlights ingredients that should be used soon. A recommendation engine matches current inventory with personal taste to suggest practical recipes, helping users cook more and waste less. For dining out, the app presents curated restaurant lists with clear menus and prices so a decision can be reached quickly. With all three capabilities combined, **What to Eat** reduces stress, saves money, and makes everyday meal planning simpler and more sustainable.

E-R Diagram

Our E-R diagram includes several key entities: Ingredient, User, Option, and Restaurant. To connect these entities and represent their interactions, we define Fridge, Recipe, and Menu as relationship sets.

- The Fridge relationship links User and Ingredient, and stores the quantity of each ingredient that a user has.
- The User entity is conceptual only. It is shown with a dashed outline in the diagram because the system does not store actual user data.
- The Recipe relationship connects Ingredient and Option, and includes details such as the quantity needed and whether the ingredient is optional for a specific recipe.
- The Option entity represents available meal choices. It stores attributes such as meal name, cuisine type, mood, and whether the meal is designed for eating out.
- The Menu relationship connects Option and Restaurant, specifying which options are available at which restaurants and at what price.
- The Restaurant entity includes attributes like restaurant ID and restaurant name.

This design structure allows us to capture both static entities (like ingredients or restaurants) and dynamic, attribute-rich relationships (like menus or recipes).

Relational Schema

Based on the E-R diagram, we created six relational tables:

- Ingredient (IngrdID, IngrdName)
- Option (OptID, MealName, EatOut, CuisineType, Mood)
- Restaurant (RstrntID, RstrntName)
- Fridge (IngrdID [FK], Qty)
- Recipe (OptID [FK], IngrdID [FK], Qty, Optional)
- Menu (RstrntID [FK], OptID [FK], Price)

Each table corresponds to either an entity or a relationship in the E-R model, with clear primary and foreign key constraints.

To ensure our schema avoids redundancy, we examined the functional dependencies and verified that the design meets the requirements of normalization:

1NF:

All attributes contain atomic values so every component in each table can't be further divided.

2NF:

All non-primary attributes are fully dependent on the entire primary key, especially in tables with composite keys like Recipe and Menu.

3NF:

There are no transitive dependencies; all non-key attributes directly depend on the primary key.

BCNF:

In addition to satisfying 3NF, all determinants in every functional dependency are candidate keys.

Our design meets BCNF because in all tables, any attribute that determines other attributes is either a primary key or part of a candidate key.

For example, in the Recipe and Menu tables, the composite primary keys fully determine all other attributes, and no partial or hidden dependencies exist.

This level of normalization ensures that our database is fully normalized, redundancy-free, and robust against anomalies in data insertion, deletion, or update.

It also supports efficient data access and future system scalability.

Integrity Constraints

1. Entity Integrity

To ensure that each table maintained uniquely identifiable records, we applied entity integrity by defining primary keys across all essential entities. Tables such as Ingredient, Option, and Restaurant were assigned an AUTO_INCREMENT primary key (e.g., IngrdID, OptID, and RstrntID) to prevent duplicate entries and enforce uniqueness. This constraint ensured that no record could exist without a valid identifier, forming the foundation of reliable relational operations across the database.

2. Referential Integrity

We enforced referential integrity by establishing foreign key constraints to maintain valid relationships between related tables. For example, in the Fridge table, IngredID is a foreign key referencing Ingredient.IngrdID, ensuring that only ingredients that exist in the Ingredient table can be added to the user's fridge. Similarly, the Recipe table links IngredID and OptID to the Ingredient and Option tables respectively. These constraints prevent the insertion of inconsistent or orphaned records, thus preserving the logical consistency between connected tables.

3. User-Defined Integrity

To address application-specific business rules, we implemented user-defined integrity constraints. One key example was ensuring that the Qty fields in the Fridge and Recipe tables always store non-negative values, reflecting realistic quantities of ingredients. Although MySQL versions may not always enforce CHECK constraints strictly, this logic was also supported at the application level through validation during data entry and API processing. This approach ensured that user inputs aligned with domain expectations and helped safeguard against invalid or illogical data entries.

Security Design

In the WhatToEat project, we implemented a basic security design by introducing role-based access control (RBAC) to manage user permissions and safeguard sensitive operations. We defined two primary user roles: Admin and General User. Admins have elevated privileges, such as the ability to add, update, or delete ingredients, recipes, and meal options, while general users can only view options and manage their own fridge contents. This role distinction was enforced through a Users table with a Role attribute and supported in the backend logic to restrict access based on role. By clearly separating responsibilities and restricting data modification rights, our system ensures secure and organized interaction with the application, laying the foundation for scalable and accountable user management.

Application Functional Design

This application is designed to assist users in managing their daily food-related decisions through digital tracking of fridge inventory, recipe-based cooking, and external meal options. The system supports core functionalities such as checking and editing fridge contents, managing ingredient consumption through recipe usage, and suggesting eating-out alternatives based on user preferences. By integrating recipe handling and meal selection into a centralized flow, the application aims to streamline meal planning, reduce food waste, and alleviate the burden of decision-making around what to eat. Each module provides clear input-output operations, ensuring a

user-friendly experience tailored to both cooking at home and exploring external dining options.

Check Ingredients

- Check recipes ingredients
 - **Input:** A specific recipe
 - **Output:** The needed ingredients
 - **Functionality:** Check the ingredients in a specific recipe
- Check ingredients in fridge
 - **Output:** All ingredients in your fridge
 - **Functionality:** Check the ingredients in your fridge

Edit Fridge

- Buy Ingredient
 - **Input :** Ingredient and its quantity
 - **Output:** The quantity of this ingredient in fridge after added in
 - **Functionality:** Add ingredient into the fridge
- Use Ingredient
 - **Input :** Ingredient and its quantity
 - **Output:** The quantity in fridge after used
 - **Functionality:** Use ingredient from the fridge
- Use Recipe
 - **Input :** A specific recipe
 - **Output:** The quantity of ingredients in fridge after used this recipe
 - **Functionality:** Use the ingredients in this recipe

Check Eatout options

- Restaurants
 - **Input:** preference
 - **Output:** All restaurants that fit the preference
 - **Functionality:** Check all restaurant options based on users' preference
- Menu
 - **Input:** A specific restaurant

- **Output:** Meal options that the restaurant serves
- **Functionality:** Check the meal options within the restaurant

Meal Options

- Available Meal Options
 - **Input:** preference
 - **Output:** All available meal options
 - **Functionality:** Check all available meal options based on users' preference
- Select Meal
 - **Input:** Preference
 - **Output:** A meal option
 - **Functionality:** Help you choose your next meal

System Implementation

- Database: mysql
- Implementation framework: Python 3.12.4 with Flask 3.1.0
- Environment control: pyenv+virtualenv
- Project co-work control: git